

Tranzactions in the FPGA Substructure

Concepts and Techniques

Abstract

What the DÆDÆLUS protocol brings to the world of transactions, within the world of FPGA-enabled Chiplet substructures for microdatacenters.

G&R Transaction Processing

Transactions & Tranzactions

We start with the original Gray and Reuter (G&R) concepts, identify the challenges and tradeoffs known at the time, and combine them with what we have since learned since.

Gray and Reuter (G&R) [1993] define a **Transaction** as:

“a sequence of one or more operations (*reads or writes*) that are executed as a single unit of work.”

Atomicity: Transactions are atomic, meaning that they are treated as a single, *indivisible* unit.

Consistency: Transactions bring the database from *one consistent state to another*.

Isolation: The execution of transactions is *isolated* from each other, so one transaction's effects are *not visible to others until it is committed*.

Durability: Once a transaction is *committed*, its *changes persist* even in the face of system failures.

Since Moss (1981) and G&R (1993) we have gained an enormous amount of experience in the application of these concepts, and they are still highly contended today.

- Locking
- Consensus
- The Physics of Entanglement

And applies/redefines these concepts for implementation in 8 port SmartNICs.

We refer to this environment as the FPGA ‘substructure’, in contrast to the x86 Host processor ‘superstructure’ in the world of Microdatacenters build out of Chiplets.

Moss: Nested SubTranzactions

Moss (1981) [3] defined Transactions as:

“collections of primitive actions that are *indivisible*. The indivisibility of transactions insures that consistent results are obtained even when requests are processed concurrently or failures occur during a request.”

In the intervening decades since the work of Moss, the industry has failed to achieve this ideal property of *indivisibility*. There is no point in trying to sharpen the Guillotine with clock synchronization (simultaneity is relative to the observer in physics), or by hunkering down and trying to force a notion of an irreversible ‘hypothetical, linearized execution that satisfies the real-time order’, as characterized by the concept of linearizability.

Nested Transactions

Provide “nested universes of synchronization and recovery from failures”.

The advantage of nested transactions over single-level transactions are that they provide concurrency control within transactions by serializing subtransactions appropriately, and that they permit parts of a transaction to fail without necessarily aborting the entire transaction.” [3].

In DAE, these ‘nested universes of synchronization’ enable us to specify ‘reversibility points’. Instead of an irreversible *smash and restart*, we use ‘invocation’ and ‘response’ as events to trigger the ancilla (control bits) to manage the forward/reverse evolution of state machines, allowing us to back out of the one way street we’ve found ourselves in.

Moss uses locking for concurrency control, and presents the necessary algorithms for locking, recovery, distributed commitment, and distributed deadlock detection for a nested transaction system.

The alternative, which has created controversy and disagreement in the database community, is to use MVCC. Which depends on snapshots.

Tranzactions are composed of any number of subtranzactions. Subtranzactions are composed of *exactly two* subtranzactions. It’s subtranzactions all the way down, until we get to the DÆ (Distributed Atomic Ethernet).

Hypertransactions improve Transactions by:

1. Using 2-way EXCHange interactions instead of individual reads or writes.
 - i. *Making sequences of read/write instructions atomic is difficult because the scheduler can interrupt the application at any time.*
 - ii. *Making operations durable is difficult because: file systems, fsync, and storage are unreliable, dramatically increasing recovery complexity [ref]*
2. Using reversible operators instead of single (one-way) operators, we explicitly use *reversible* operators on subtransactions.
 - i. Extends Moss’ nested subtransactions by enabling a larger repertoire of hazards to be addressed without having to abort (smash & restart)
3. Eliminating Network Partitions, and CAP Theorem Tradeoffs.
4. Replacing Locking & 2PC (the anti-availability protocol) with *Hyperlocking* — using *Alternating Causality* (AC) to eliminate race conditions, and *Ownership* to enable ‘determinism on demand’ without sacrificing fairness.

5. By nesting reversibility and transactionality both across the Ethernet interface, and the PCIe/CXL Bus.

What ChatGPT Says:

Jim Gray and Andreas Reuter's refer to 'isolation' as the need to ensure that concurrent execution of multiple transactions does not compromise the consistency and correctness of the database. The isolation property, one of the ACID properties (Atomicity, Consistency, Isolation, Durability), guarantees that the execution of one transaction is isolated from the effects of other concurrent transactions until it is committed. Here are some key concepts related to isolation:

1. Concurrency Control:

- Concurrency control mechanisms are employed to manage the simultaneous execution of transactions in a multi-user environment.
- Locking: Transactions acquire locks on data items to prevent other transactions from accessing the same items concurrently. This ensures that only one transaction can modify a data item at a time.
- Two main types of locks are shared locks (for read operations) and exclusive locks (for write operations).

2. Isolation Levels:

- Isolation levels define the degree to which the operations of one transaction are isolated from the operations of other concurrently executing transactions.
- The four standard isolation levels defined by the SQL standard are:
 - Read Uncommitted: Transactions can read uncommitted changes made by other transactions.
 - Read Committed: Transactions can only read committed changes made by other transactions.
 - Repeatable Read: A transaction can read the same data multiple times, and the result will not change until the transaction is committed.
 - Serializable: The highest level of isolation, ensuring that the execution of transactions is equivalent to some serial order.

3. Phantom Read and Non-Repeatable Read:

- Phantom Read: A phenomenon where a transaction sees a set of rows that match a certain condition, but the set changes between the transaction's reads due to another transaction's insert or delete operations.
- Non-Repeatable Read: A phenomenon where a transaction reads the same data multiple times, but the data changes between the reads due to another transaction's update or delete operations.

4. Snapshot Isolation:

- Snapshot isolation is a higher-level isolation level that provides a consistent snapshot of the database to each transaction.
- Each transaction sees a snapshot of the database as of the start of the transaction, and changes made by other transactions after the start of the current transaction are not visible.

5. Serializable Schedule:

- A schedule of transactions is considered serializable if it produces the same result as some serial execution of the transactions.
- Serializable schedules ensure a high level of consistency but may lead to performance overhead due to the need for strict ordering of transactions.

6. Optimistic Concurrency Control:

- In addition to locking, optimistic concurrency control is an approach where transactions proceed with their execution assuming that conflicts are unlikely.
- Conflicts are checked at the end of the transaction, and if conflicts are detected, appropriate actions are taken (e.g., rolling back one of the conflicting transactions).

Understanding and implementing isolation concepts are crucial for designing efficient and correct transaction processing systems, and the choice of isolation level depends on the specific requirements of the application and the trade-offs between consistency and performance.